PROJECT DOCUMENTATION / 05 NOVEMBER 2026

Ollie

Document RAG Assistant

Project in one sentence
A React chat interface backed by FastAPI that extracts text from uploaded PDFs, images, and TXT files, embeds overlapping chunks, retrieves relevant passages, and sends compact grounded context to a language model.

COMPLETE TECHNICAL GUIDE

Architecture, ingestion, chunking, embeddings, retrieval, token use, models, security, limitations, benefits, and Netlify deployment readiness.

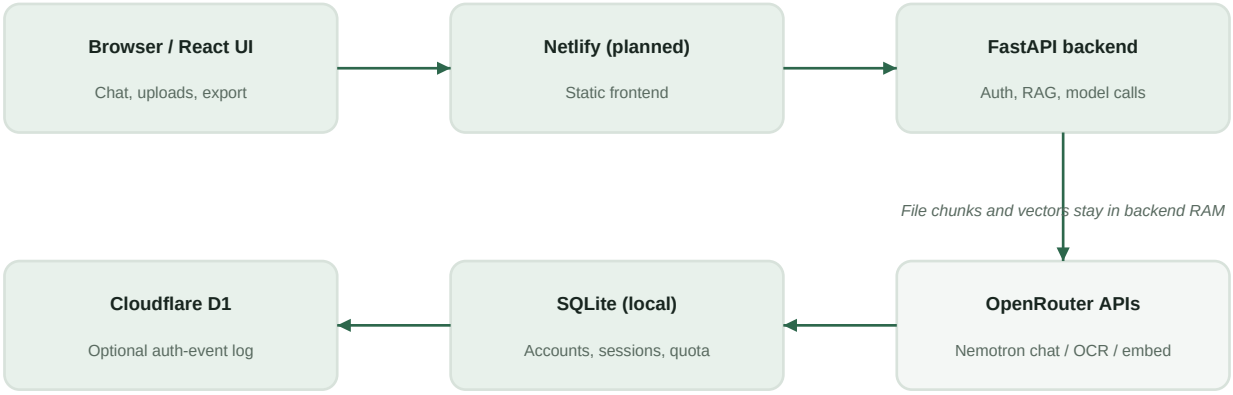
FRONTEND	BACKEND	CHAT / OCR	EMBEDDINGS
React 19 + Vite 8	Python 3.13 + FastAPI	Nemotron 3.5 Super VL	Nemotron 3 Embed 1B

Prepared from the project source and local model configuration. No API keys, tokens, passwords, or user records are included.

1. What Ollie does

Ollie is a retrieval-augmented generation (RAG) application. RAG does not retrain the model: it finds passages in currently uploaded files and includes those passages in the model prompt. This helps answer questions about material the base model may not know.

System architecture



Normal flow: browser -> React UI -> FastAPI API -> optional query embedding and local vector ranking -> OpenRouter chat completion -> answer and source references return to the browser. SQLite stores user/account state. Optional Cloudflare D1 logging stores login/logout audit events. Uploaded document chunks and vectors currently live in backend memory.

Component responsibilities

Layer	Responsibility
React frontend	Signup/login, chat UI, upload/drop area, AI Think toggle, usage meter, upload progress, collapsible source panel, print-to-PDF export.
FastAPI backend	Validates users/files; extracts text; chunks content; calls embedding/chat APIs; retrieves passages; enforces quotas.
SQLite	Stores user records, salted password hashes, terms acceptance, session-token hashes, and UTC daily question counts.
OpenRouter	OpenAI-compatible API endpoint routing requests to configured chat, OCR, and embedding models.
Cloudflare D1 (optional)	Audit records only: email, event type, event ID, UTC timestamp. It is not the account database.

Netlify target
The built React/Vite assets can be deployed as a static Netlify site. Production API requests still need routing to a separate Python API host, and persistent account storage is required before the complete application can be publicly used.

2. Upload and document processing

The upload area accepts PDF, PNG/JPG/JPEG/WEBP image, and UTF-8 TXT files. The browser sends each file to POST /api/documents. That API call completes only after text extraction, chunking, and embedding calls finish, so the interface can show processing feedback until indexing is complete.

Processing sequence

- Validate session, extension, image signature, file size, per-user file count, and upload rate limit.
- Extract PDF text with pypdf. Pages yielding fewer than 30 characters are considered likely scans and sent one page at a time for OCR.
- Send image files to the configured vision model for transcription. Decode TXT as UTF-8 (including BOM).
- Normalize whitespace and split extracted text into chunks. Keep each PDF chunk paired with its page number; image and TXT use page marker 1.
- Send chunk text to the embedding model in batches of up to 32 inputs. Pair each returned vector with its original text chunk.
- Keep file metadata, chunks, and vectors in backend memory, then return file details to the frontend.

File limits in current backend

Input	Current limit
PDF	25 MB; 1-300 pages; at most 40 likely scanned pages requiring OCR. Password-protected PDFs are rejected.
PNG/JPG/JPEG/WEBP	10 MB per image. Actual file signature is checked before OCR.
TXT	2 MB; UTF-8 required.
All types	Seven total per user session; maximum 1,500 chunks per file; 10 uploads per user per hour.

Restart caveat
Uploaded files and their vectors are stored in process memory, not SQLite or D1. A backend restart/redeploy clears indexed files; users have to upload again. Hosted use needs durable file/index storage or a clear re-index strategy.

3. Chunking and embeddings

Chunking is character-based, not tokenizer-based or semantic-boundary-based. Whitespace is collapsed, then each page is split into target windows of 900 characters. Before a cut, the splitter searches for a space from the midpoint of that window to its end, so it often cuts at a word boundary. The next window starts 100 characters before the prior window ends.

Simplified view

```
Normalized page text -> [chunk 1: chars 0-900] -> [chunk 2: starts about 100 chars before chunk 1 ends] ->
[chunk 3] -> ...
```

Overlap keeps context that sits near a split. A definition ending one chunk and its explanation beginning in the next may then remain jointly retrievable. The trade-off is duplicated text and some additional embedding use. Actual chunk lengths vary slightly due to the boundary search.

Embedding generation

For each chunk, Ollie sends its text to OpenRouter /embeddings using nvidia/nemotron-3-embed-1b:free. The model converts text into a dense numeric vector. Related meanings can be close in vector space even when exact words differ. Ollie does not hard-code the vector dimension; it stores the returned values.

Vectors are generated at upload time and reused while the server session retains the index. For a document-grounded question, Ollie also embeds the question and compares it with the stored chunk vectors.

Choice	Benefit	Trade-off
900-character chunks	Focused passages for retrieval.	A concept and distant explanation can be split apart.
100-character overlap	Preserves nearby context across chunk edges.	Some content is repeated and embedded.
Prefer space boundary	Avoids cutting inside a word when possible.	Chunk sizes vary a little.
Batch of 32 inputs	Reduces per-request overhead.	Provider throttling still applies.
Vectors in RAM	Simple prototype; no vector database needed.	Not durable/shared; search scans chunks linearly.

> **Why embeddings help**
> Keyword search needs matching words. Embeddings can retrieve a passage about "high implementation costs" for a question phrased as "What makes this expensive?"

4. Retrieval and answer generation

With AI Think off and documents attached, Ollie embeds the question. It checks only chunks for documents owned by the signed-in user and verifies document IDs on the server. It ranks those chunks using cosine similarity: vector dot product divided by the product of vector lengths.

Chunks scoring above 0.15 are eligible. Ollie sorts eligible passages by score and selects at most the four highest. Combined retrieved context is capped at 6,000 characters. This is a direct in-memory scan rather than a vector database index.

The selected passages are placed in a system prompt telling the model to treat file excerpts as untrusted reference data, answer from supplied evidence, cite exact filenames and PDF pages, and admit when evidence is insufficient. Images and TXT use filename citations; PDFs include page numbers.

Prompt limits

Prompt part	Current setting
Retrieved document context	At most 4 chunks, at most 6,000 combined characters including source labels.
Recent history	Last 6 turns, each content string truncated to at most 1,200 characters.
Question	Backend accepts 1-8,000 characters.
Generation	temperature 0.2; max_tokens 700; request/response is non-streaming.

Two modes

Mode	Documents used?	Query embedding?	Behavior
AI Think OFF	Yes, when relevant documents are attached.	Yes, for semantic retrieval.	Grounded answer and source references; can report insufficient evidence.
AI Think ON	No. Backend excludes uploaded documents.	No document retrieval embedding.	General model answer; can be stale; no web browsing.

5. Tokenization and token efficiency

A model token is a tokenizer unit: it can be a word, word piece, punctuation, or other unit. Ollie does not run the Nemotron tokenizer locally or measure exact prompt tokens before requests. Its chunk and context limits are mostly character-based. Therefore 900 characters is not a fixed number of tokens; token counts vary with language, vocabulary, punctuation, and formatting.

Where model tokens are used

Stage	Input / output	When
OCR / image transcription	Image or scanned page plus extraction prompt; image OCR output is capped at 4,000 tokens.	Only for image inputs or scanned PDF pages.
Embedding at ingestion	Extracted text chunks, sent in batches of up to 32.	Once during successful upload; overlap repeats some text.
Query embedding	Question text.	Each document-grounded question with files attached.
Chat input	System instruction, retrieved excerpts, up to six prior turns, and current question.	Every chat request.
Chat output	Up to 700 tokens. AI Think requests up to 300 reasoning tokens with exclude=true.	Every chat request; provider accounting applies.

How context is saved

- Files are extracted, chunked, and embedded at upload instead of resending the entire source for every question.
- RAG sends the question for embedding, ranks existing vectors, and forwards only a few high-ranked chunks.
- AI Think skips document retrieval and query embedding.
- History and answer length have explicit limits.

The daily limit is not a token allowance
The 100-per-UTC-day quota counts chat requests per account, not model tokens. It does not limit OCR/embedding tokens or guarantee an API cost. Ollie reserves quota when a chat request begins, so a provider failure can still consume one question.

Costs and latency depend on model/provider choice, file size and number of chunks, OCR requirements, and service throttling. Free model access and limits can change.

6. Models, APIs, and fine-tuning

Purpose	Configured model	Use
Chat and AI Think	nvidia/nemotron-3.5-super-vl:free	Generate grounded or independent answers.
Embeddings	nvidia/nemotron-3-embed-1b:free	Vectorize chunks and RAG questions.
OCR / vision	nvidia/nemotron-3.5-super-vl:free	Read text in uploaded images and scanned pages.
API gateway	OpenRouter, https://openrouter.ai/api/v1	Routes chat and embedding requests to configured models.

These model IDs are in the local backend environment settings. The Python source has Qwen names as fallback defaults if those values are missing; the active local settings override them with NVIDIA models. This PDF intentionally omits key values.

AI Think prompt tuning

> **Response quality instructions**
>
> AI Think uses a refined server-side prompt: answer the main question first, match the requested depth, organize complex replies into bullets or steps, and use examples when helpful. It flags assumptions and uncertainty, avoids invented sources, and keeps private reasoning hidden. This is prompt engineering; hosted model weights are unchanged.

OCR prompts request transcription only. Image OCR uses temperature 0 and a 4,000-token response cap; scanned PDF pages are processed one page per request. Chat uses temperature 0.2 and a 700-token response cap. AI Think asks for a 300-token reasoning maximum with exclude=true. Provider semantics and billing may differ; these are request parameters, not guaranteed billing ceilings.

Tools and languages

Area	Technology / version in project
Frontend	JavaScript / JSX, React 19.3.1, Vite 8.3.6, CSS, Lucide React 0.468.0.
Backend	Python 3.13, FastAPI 0.162.0, Uvicorn 0.54.0, HTTPX 0.28.1.
Document / config	pypdf 6.21.4, python-multipart 0.0.33, python-dotenv 1.2.4.
Storage	SQLite for accounts/session/usage; in-memory Python objects for document index; optional Cloudflare D1 REST API for audit records.
Local architecture	Vite development server with /api proxy to FastAPI on port 8000.

7. Security, privacy, and limits

Controls currently implemented

- Passwords are stored as salted PBKDF2-HMAC-SHA256 hashes with 600,000 iterations and random 16-byte salts; verification uses constant-time comparison. Plaintext passwords are not stored.
- Session IDs are random. The cookie is HttpOnly, SameSite=Lax, path-scoped, and expires in seven days; SQLite stores the token hash. Secure-cookie mode is controlled by an environment variable and is false for local HTTP.
- Signup accepts Gmail addresses only; passwords need 10-128 characters with lowercase, uppercase, digit, and symbol.
- OpenRouter and Cloudflare credentials are loaded in the backend, not React. Local .env is ignored by Git. Do not commit or publish it.
- CORS uses an allowed-origin list and write/delete requests check Origin when supplied. Security headers include nosniff, frame denial, referrer policy, and permissions policy.
- In-memory rate limits cover signup, login, uploads, and chat. These counters reset when the process restarts.
- RAG prompt treats file content as untrusted reference data, not as instructions.

Data locations

Data	Current location	Limit
Accounts, hashes, sessions, quota	SQLite at OLLIE_DB_PATH or backend fallback.	Must be on persistent storage and backed up in hosting.
Files, text chunks, vectors	Backend memory.	Restart clears them; text is sent to OpenRouter for model features.
Auth audit events	Optional Cloudflare D1.	Email, event type, event ID and UTC time only; not passwords.
OpenRouter key	Backend environment / local .env.	Keep private; free endpoints may throttle or log data per provider terms.

Before unrestricted public launch

Deploy the frontend to Netlify and run FastAPI on a separate Python-capable host. Migrate local SQLite account/session/usage state to reliable hosted persistence or durable storage. Enable secure cookies behind HTTPS, set the production API URL and allowed origin, store API keys in the backend host secret manager, enable backups, and add email verification and public-registration abuse prevention. Decide how file indexes and rate limits survive backend restarts or multiple instances.

> **Local storage and privacy**
>
> In local demo mode, account records, session hashes, and daily counts are stored in the configured SQLite database; theme preference is stored in browser localStorage. Uploaded files, extracted chunks, and embeddings stay in backend memory and clear on restart. File content is still sent to OpenRouter for OCR, embeddings, and answers, so this is local-first storage, not offline processing. Optional Cloudflare D1 stores email and login/logout event times only. Do not upload confidential files.

8. Benefits and time-saving estimate

Useful scenarios

- Students can ask about lecture notes, readings, and course PDFs, then check cited source pages.
- Researchers can find definitions, comparisons, and supporting evidence across a small file collection.
- Teams can query policies, manuals, technical notes, or transcripts without repeatedly searching each file.
- Images and screenshots are included through OCR; UTF-8 text notes are indexed directly.
- AI Think offers general model knowledge when independent explanation is wanted; turning it off returns to document retrieval.

Efficiency advantages

Manual workflow	Ollie workflow
Open documents, search terms, scan surrounding pages, compare sources, and write an answer manually.	Upload and index once; ask in natural language; Ollie ranks passages and returns an answer with source references.
Reread a file for each question.	Reuse chunk embeddings until the backend process resets.
Harder to trace an answer to a page.	PDF retrieval keeps filename and page number with chunks for verification.

Illustrative time example - not a measured benchmark

Assume locating, reading, and synthesizing one answer across several long files manually takes 10-20 minutes. Assume an Ollie pass takes 2-5 minutes, including reading the answer and checking citations. Under these assumptions, the estimated saving is about 5-18 minutes per question (roughly 50-90% for that task). Ten similar questions might save 50-180 minutes. These are planning estimates only, not measured results. Actual time varies by source quality, OCR, API latency, question complexity, and verification effort.

To publish a real portfolio metric, time a fixed set of representative questions manually and with Ollie, include verification in both, then calculate: (manual minutes - Ollie minutes) / manual minutes x 100.

> **Quality boundary**
> Ollie reduces the time spent locating and organizing passages; it does not remove verification. OCR may miss text, retrieval may miss a relevant chunk, and a model may misread evidence.

9. Local run and Netlify plan

Local run commands

Run these from the project folder in separate PowerShell windows. Vite proxies /api requests to FastAPI on local port 8000.

```
Backend: .tools\python\python.exe -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
Frontend: .tools\node\node.exe node_modules/vite/bin/vite.js --host 127.0.0.1 --port 5174
Open: http://127.0.0.1:5174
```

Netlify deployment pieces

Part	Plan	Required preparation
React frontend	Netlify static site; npm run build; publish dist.	Route /api to backend via Netlify proxy/rewrite or add API-base configuration; add SPA fallback if needed.
FastAPI	Separate Python-capable backend host.	HTTPS, secure cookies, correct CORS/origin, API secret configuration.
Accounts	Hosted persistent database or durable disk.	Local SQLite path is not automatically durable or portable.
File index	Persistent vector storage or re-upload/re-index plan.	Current chunks and embeddings are in process memory.
Audit log	Optional Cloudflare D1.	D1 is only for login/logout events in current code; keep token private.

The current frontend uses relative /api paths and Vite proxies them only in local development. Configure a Netlify proxy/rewrite to the separate backend or add a production API-base setting before chat/auth can work online. Do not upload backend/.env to GitHub, Netlify artifacts, or this report. Store keys only in private backend host settings.

Resume-ready summary

> **Project description**
> Built Ollie, a React and FastAPI RAG assistant that accepts up to seven PDFs, images, or TXT files, extracts/OCRs text, creates overlapping chunks, embeds them with NVIDIA Nemotron through OpenRouter, ranks semantic matches with cosine similarity, and generates source-aware answers. Added independent AI Think mode, password hashing and secure sessions, per-user daily quota, upload progress, and print-to-PDF chat export.

Project files reviewed

backend/main.py, backend/.env (secret values redacted), backend/requirements.txt, src/App.jsx, src/styles.css, package.json, package-lock.json, vite.config.js, and README.md. Prepared 5 November 2026.

This guide describes the local project as implemented. Model access, provider limits, and free-tier terms can change. API credential values are intentionally excluded.